

Производственная практика: научно-исследовательская работа
ОСНОВЫ РАБОТЫ С РЕЛЯЦИОННЫМИ БАЗАМИ ДАННЫХ С
ИСПОЛЬЗОВАНИЕМ ЯЗЫКА Python

Москва, 2025

Оглавление

РАБОТА С ДАННЫМИ SQLite В ПРОГРАММАХ НА ЯЗЫКЕ Python	3
2.1. Подготовительные операции	3
2.2. Создание БД bd1.sqlite с 2 таблицами: stud и sessija.....	4
2.3. Запись данных в таблицы stud и sessija	5
2.4. Множественная вставка данных в таблицы БД	7
2.5. Чтение и отображение содержимого одной из таблиц в БД.	7
2.6. Чтение данных из таблицы БД.	9
2.7. Получение списка таблиц в составе БД.	10
2.8. Считывание информации о структуре таблицы, имеющейся в БД.	11
2.9. Ввод данных по запросу с клавиатуры и занесение их в таблицу.	12
ЗАДАНИЕ ДЛЯ ЭТАПА 3 НИР	14
3.1. Цель этапа.....	14
3.2. Содержание этапа	14
3.3. Создание таблицы, открытие файла и организация меню управления базой данных.	14
3.4. Функции.....	17
3.5. Работа программы.	21
3.6. Полноценный код программы.	24
ЗАДАНИЕ ДЛЯ ЭТАПА 4 НИР	29
4.1. Цель этапа.....	29
4.2. Структура БД.....	29
4.2. Описание алгоритма по 1 заданию.	32
4.3. Описание алгоритма по 2 заданию.	35
4.4. Результаты тестирования.	38
4.5. Полноценный код программы.	44
Вывод.....	56

ЭТАП 2 НИР

РАБОТА С ДАННЫМИ SQLite В ПРОГРАММАХ НА ЯЗЫКЕ Python

2.1. Подготовительные операции

Перед началом работы в среде **Python** с БД **SQLite** проверили:

- наличие библиотеки **sqlite3.dll**
- наличие подкаталога (пакета) **sqlite3**

Код:

```
import sqlite3
import os
print("SQLite версия:", sqlite3.sqlite_version)
lib_path = os.path.dirname(sqlite3.__file__)
print("Каталог существует:", os.path.exists(lib_path))
```

Вывод:

SQLite версия: 3.50.4

Каталог существует: True

Импортировали модуль **sqlite3** для работы с SQLite.

Код:

```
import sqlite3
```

Дальнейшие действия будут содержать типовые операции с базами данных, применимые в программах на **Python**. А именно:

- создание базы данных и входящих в нее таблиц;
- занесение записей в таблицы;
- удаление записей;
- изменение структуры таблиц (удаление или добавление полей, изменение их параметров); – изменение значений в заданных полях и записях.

Для созданной БД понадобятся:

- получение списка таблиц в составе БД;
- считывание информации о структуре имеющихся в БД таблиц (имена и параметры входящих в таблицы полей).

2.2. Создание БД `bd1.sqlite` с 2 таблицами: `stud` и `sessija`.

Для начала создадим БД для хранения информации о студентах, обучающихся по некоторому направлению (таблица ***stud***), и об их успеваемости (таблица ***sessija***).

Установим соединение с существующей БД или с формирования новой БД с созданием переменной **con**:

```
con=sqlite3.connect('bd1.sqlite')
```

*Данная команда создает подключение к базе данных SQLite. Если файл **bd1.sqlite** существует, он открывается для работы. Если файл отсутствует, он автоматически создается как новая пустая база данных. Возвращает объект соединения **con**, который используется для выполнения всех последующих операций с БД.*

Создадим переменную-курсор с некоторым именем, например, **cur**:

```
cur=con.cursor()
```

Курсор нужен для выполнения всех операций с базой данных: он выполняет SQL-запросы, читает результаты выборки и обеспечивает навигацию по полученным данным.

Создадим переменную с символьной строкой, содержащей скрипт с командами формирования таблиц:

```
sql="""\n\nCREATE TABLE stud (id TEXT PRIMARY KEY,\n\nfio TEXT, email TEXT);\n\nCREATE TABLE sessija (id TEXT SECONDARY KEY,\n\nDisz TEXT, Ozenka INTEGER, Prepod TEXT, Dat TEXT)\n\n"""
```

Обратный слэш отменяет автоматический перевод строки, который Python добавляет после открывающих тройных кавычек. Это позволяет SQL-запросу начинаться сразу с первой строки, без пустых отступов.

Выполним совокупность SQL-команд (скрипт) из этой переменной. :

`cur.executescript(sql)` - выполняет несколько *SQL*-команд за один раз, разделенных точкой с запятой.

Закроем курсор и БД, так как это необходимо для сохранения всех изменений в файл БД, освобождения занятой памяти и разблокировки файла для работы других программ, предотвращая потерю данных и утечку ресурсов:

```
cur.close()
```

```
con.close()
```

База данных создана, структура настроена, но данные отсутствуют.

(см. Рисунок 1 – Рисунок 3)

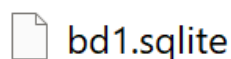


Рис. 1. Создание файла с данными.

TABLES		id	Disz	Ozenka	Prepod	Dat
> sessija		Filter	Filter	Filter	Filter	Filter
> stud	1					

Рис. 2. Отсутствие данных в таблице sessija.

TABLES		id	fio	email
> sessija		Filter	Filter	Filter
> stud	1			

Рис. 3. Отсутствие данных в таблице stud.

2.3. Запись данных в таблицы stud и sessija

Установим соединение с ранее созданной БД:

```
con=sqlite3.connect('bd1.sqlite')
```

Создадим курсор:

```
cur=con.cursor()
```

Создание переменной с текстом SQL-команды вставки записи в таблицу stud:

```
sql="""\nINSERT INTO stud (id,fio,email) VALUES\n('0020233240','Зеленкина Кс. М.','ZelenkinaXM@mpei.ru')\n"""
```

Выполнение этой SQL-команды:

```
cur.execute(sql)
```

В отличие от executescript, execute – выполняет один SQL-запрос.

Сохранение выполнения команды:

```
con.commit()
```

Выполнение аналогичных операций для вставки еще одной записи в таблицу stud:

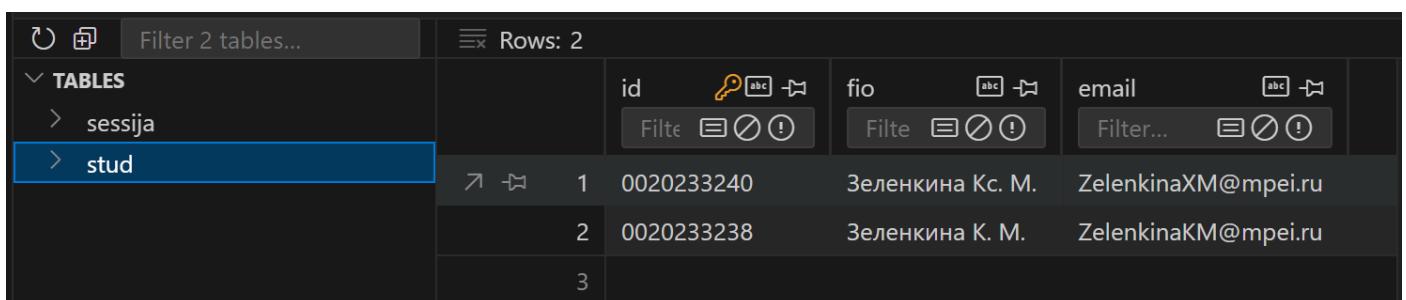
```
sql="""\nINSERT INTO stud (id,fio,email) VALUES\n('0020233238','Зеленкина К. М.','ZelenkinaKM@mpei.ru')\n"""
```

```
cur.execute(sql)\ncon.commit()
```

Закрытие курсора и БД.

```
cur.close()\ncon.close()
```

Данные добавились в таблицу stud (см. Рисунок 4)



The screenshot shows a database management interface. On the left, a sidebar lists tables: 'sessija' and 'stud'. The 'stud' table is selected and highlighted in blue. The main area displays the data of the 'stud' table. It shows two rows of data. The first row has id '0020233240', fio 'Зеленкина Кс. М.', and email 'ZelenkinaXM@mpei.ru'. The second row has id '0020233238', fio 'Зеленкина К. М.', and email 'ZelenkinaKM@mpei.ru'. The table has columns 'id', 'fio', and 'email'. The interface also shows a 'Filter 2 tables...' button at the top left and a 'Rows: 2' indicator at the top right.

	id	fio	email
1	0020233240	Зеленкина Кс. М.	ZelenkinaXM@mpei.ru
2	0020233238	Зеленкина К. М.	ZelenkinaKM@mpei.ru

Рис. 4. Добавление данных в таблицу stud.

2.4. Множественная вставка данных в таблицы БД

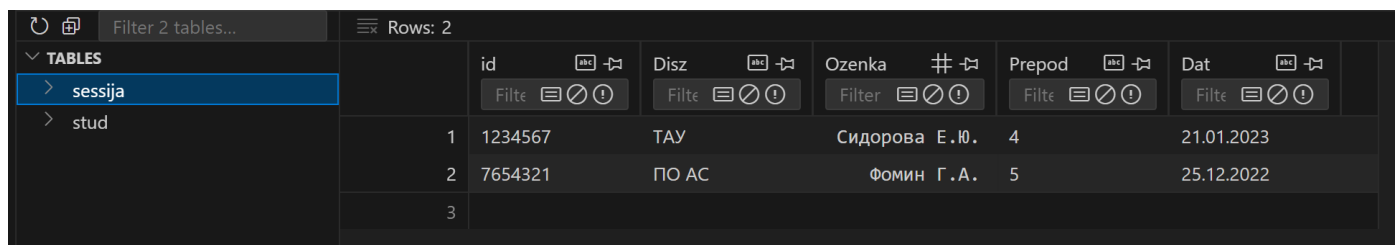
Создадим список с двумя кортежами – для записи в таблицу **sessija**. Здесь в каждом кортеже первый элемент – это символьная строка с номером студенческого билета, затем – фамилия преподавателя и полученная оценка.

```
arr=[('1234567', 'ТАУ', 'Сидорова Е.Ю. ', 4, '21.01.2023'),  
( '7654321', ' ПО АС', 'Фомин Г.А. ', 5, '25.12.2022')]  
con=sqlite3.connect('bd1.sqlite')  
cur=con.cursor()  
sql="""\n  
INSERT INTO sessija (id, Disz, Ozenka, Prepod, Dat)  
VALUES (?, ?, ?, ?, ?)  
"""
```

После ключевого слова **VALUES** число знаков вопроса должно равняться числу записываемых полей. Выполнение «множественной» команды вставки строк из **arr**:

```
cur.executemany(sql, arr)  
con.commit()
```

В результате в таблицу **sessija** вставлены 2 строки с данными из кортежей в **arr**:



The screenshot shows a database management interface. On the left, a sidebar lists tables: 'sessija' and 'stud'. The 'sessija' table is selected. The main area displays the table's contents. The table has five columns: 'id', 'Disz', 'Ozenka', 'Prepod', and 'Dat'. There are two rows of data. The first row has values: 1, 1234567, ТАУ, Сидорова Е.Ю., 4, 21.01.2023. The second row has values: 2, 7654321, ПО АС, Фомин Г.А., 5, 25.12.2022. A third row is partially visible with '3' in the 'id' column.

	id	Disz	Ozenka	Prepod	Dat
1	1234567	ТАУ	Сидорова Е.Ю.	4	21.01.2023
2	7654321	ПО АС	Фомин Г.А.	5	25.12.2022
3					

Рис. 5. Добавление данных в таблицу *sessija*.

Закрытие курсора и БД:

```
cur.close()  
con.close()
```

2.5. Чтение и отображение содержимого одной из таблиц в БД.

В начале необходимо создать программу **test3.py**, позволяющую увидеть содержимое указанной таблицы в заданной БД, которую следует записать в файл **test3.py**:

```

import os,sqlite3
def select_cmd():
#просмотр таблицы с именем в tblname из БД с именем в
dbname
    sql = 'SELECT * FROM {}'.format(tblname)
    with con:
        data = cur.execute(sql).fetchall()
    return (data)
dbname=' '
###Выбор БД с проверкой ее наличия
while (os.path.isfile(dbname)!=True):
    dbname = input('Укажите имя файла SQLite: ')
    if (os.path.isfile(dbname)==True): break
    print('Нет такого файла!')
tblname = input('Укажите имя таблицы: ') #ввод имени
con = sqlite3.connect(dbname)
cur = con.cursor()
dan=select_cmd()
nzap=len(dan)
print('Таблица: ',tblname,' из БД ',dbname)
for i in range(nzap):
    print(dan[i])
cur.close()
con.close()

```

Запустим на выполнение программу **test3** и введём имена ранее созданной БД **bd1.sqlite** и одной из таблиц – *stud* или *sessija*.

Вывод таблицы stud:

```

Укажите имя файла SQLite: bd1.sqlite
Укажите имя таблицы: stud
Таблица: stud из БД bd1.sqlite
('0020233240', 'Зеленкина Кс. М.',
'ZelenkinaXM@mpei.ru')
('0020233238', 'Зеленкина К. М.',
'ZelenkinaKM@mpei.ru')

```


Вывод таблицы *sessija*:

Укажите имя файла SQLite: bd1.sqlite

Укажите имя таблицы: sessija

Таблица: sessija из БД bd1.sqlite

('1234567', 'ТАУ', 'Сидорова Е.Ю. ', '4', '21.01.2023')

('7654321', ' ПО АС', 'Фомин Г.А. ', '5', '25.12.2022')

Указание несуществующего файла:

Укажите имя файла SQLite: test.sqlite

Нет такого файла!

Укажите имя файла SQLite:

Указание несуществующей таблицы:

Укажите имя файла SQLite: bd1.sqlite

Укажите имя таблицы: test

sqlite3.OperationalError: no such table: test

Программа вывела данные таблиц *stud* и *sessija*. При введении несуществующего файла вывела предупреждение, при несуществующей таблице - ошибку.

2.6. Чтение данных из таблицы БД.

Пусть имеется БД **bd1.sqlite**, содержащая таблицу *stud*. Получим содержимое всех записей этой таблицы.

Откроем БД и создадим курсор:

```
con=sqlite3.connect("bd1.sqlite")
```

```
cur=con.cursor()
```

Выполним SQL- команды для чтения всего содержимого указанной таблицы из БД в курсор:

```
cur.execute("SELECT * FROM stud")
```

Запишем содержимое курсора в список с указанным именем (элементы списка – кортежи со значениями полей в записях):

```
ar=cur.fetchall()
```

И, как всегда, закроем курсор и соединения с БД:

```
cur.close()
```

```
con.close()
```

Отображение полученного списка **ar**:

```
print(ar)
```

Вывод:

```
[('0020233240', 'Зеленкина Кс. М.', 'ZelenkinaXM@mpei.ru'),  
( '0020233238', 'Зеленкина К. М.', 'ZelenkinaKM@mpei.ru')]
```

2.7. Получение списка таблиц в составе БД.

Программы могут работать с разными базами данных (подобно тому, как они могут работать с разными файлами). Поэтому важно уметь в программе определить структуру БД, с которой она начинает работу.

На этапе 4 потребуется работать с БД **VUZ.sqlite**. Для того, чтобы определить, какие таблицы входят в состав этой БД следует выполнить такую последовательность операторов:

Откроем БД и установим связь с **con**:

```
import sqlite3
```

```
con=sqlite3.connect('VUZ.sqlite')
```

Создадим курсор:

```
cur = con.cursor()
```

Сформируем переменную с SQL-запросом на выборку данных о структуре БД:

```
sql="""\
```

```
SELECT name FROM sqlite_master WHERE type IN
```

```
('table','view') AND name NOT LIKE 'sqlite_%' UNION ALL
```

```
SELECT
```

```
name FROM sqlite_temp_master WHERE type IN ('table','view')
```

```
ORDER BY 1;
```

```
"""
```

Выполним запрос:

```
cur.execute(sql)
```

Сохраним результат запроса:

```
con.commit()
```

Выгрузим все данные из выполненного SELECT-запроса в виде списка кортежей.

```
cur.fetchall()
```

Закрываются курсор и БД:

```
cur.close()
```

```
con.close()
```

2.8. Считывание информации о структуре таблицы, имеющейся в БД.

Пусть рассматривается та же БД **VUZ.sqlite**. Требуется получить список имен всех полей входящей в БД таблицы *vuzkart*.

Для этого следует выполнить следующие операторы:

Создание соединения с БД:

```
con=sqlite3.connect('VUZ.sqlite')
```

Создание вспомогательной функции:

```
def my_factory(c,r):
```

```
    ###Функция для извлечения имен полей
```

```
    d={}
```

```
    for i,name in enumerate(c.description):
```

```
        d[name[0]]=r[i]
```

```
        d[i]=r[i]
```

```
    return(d)
```

Эта функция записывается в атрибут *row_factory* объекта *con*:

```
con.row_factory=my_factory
```

Создаётся курсор:

```
cur=con.cursor()
```

Исполняется SQL запрос (символьная строка с запросом записана в аргумент метода *execute*):

```
cur.execute('SELECT * FROM vuzkart')
```

В объект **ar** (словарь) копируются данные из первой записи в таблице БД:
`ar=cur.fetchone()`

Закрываются курсор и БД:

```
cur.close()
con.close()
```

В объект *fld_names* записывается список имен полей таблицы:
`fld_names=list(ar.keys())[::2]`

Отобразим на экране:
`print(fld_names)`

Вывод:

```
['codvuz', 'z1', 'z1full', 'z2', 'z2ustav', 'foundyear',
'z8', 'z9', 'z12', 'e_mail', 'www', 'z15', 'region',
'city', 'status', 'oblname', 'gr_ved', 'prof']
```

2.9. Ввод данных по запросу с клавиатуры и занесение их в таблицу.

При выполнении этих операций будем использовать ранее созданную БД **bd1.sqlite** и данные еще одного студента.

```
import sqlite3
con=sqlite3.connect('bd1.sqlite')
cur=con.cursor()
ar=[]; vv=[]
vv.append(input('Номер студенческого билета='))
vv.append(input('ФИО студента = '))
vv.append(input('e-mail студента ='))
```

Введенные данные оформляются в виде кортежа и присоединяются к списку **ar** с использованием метода **append()**:
`ar.append(tuple(vv))`

Для занесения новой строки в таблицу **stud** создадим и исполним SQL-запрос:
`sql='INSERT INTO stud (id,fio,email) VALUES (?, ?,?)'`
`cur.executemany(sql,ar)`
`con.commit()`

Теперь строка занесена в таблицу. Отобразим все ее содержимое (см. Рисунок 6).
`cur.execute('SELECT * FROM stud')`
`ar1=cur.fetchall()`

```
print(ar1)
cur.close()
con.close()
```

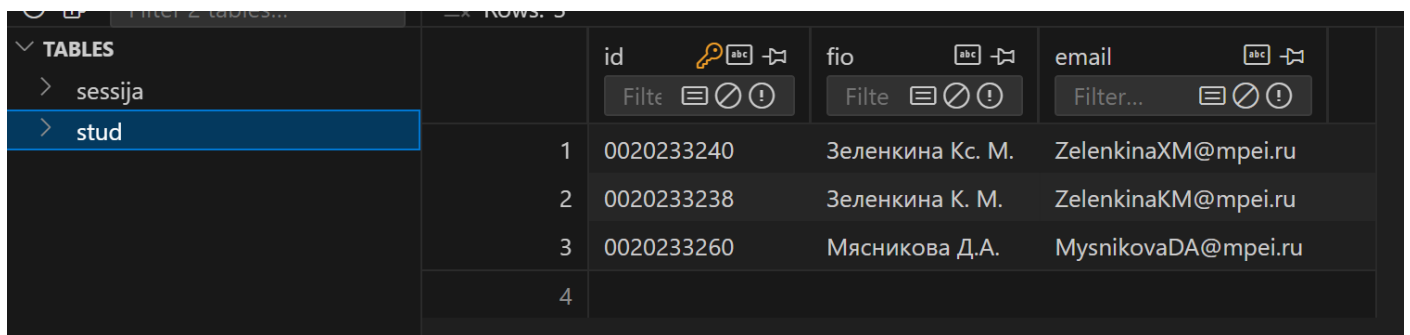
Вывод:

Номер студенческого билета=0020233260

ФИО студента = Мясникова Д.А.

e-mail студента =MysnikovaDA@mpei.ru

```
[('0020233240', 'Зеленкина Кс. М.', 'ZelenkinaXM@mpei.ru'),
 ('0020233238', 'Зеленкина К. М.', 'ZelenkinaKM@mpei.ru'),
 ('0020233260', 'Мясникова Д.А.', 'MysnikovaDA@mpei.ru')]
```



	id	fio	email
1	0020233240	Зеленкина Кс. М.	ZelenkinaXM@mpei.ru
2	0020233238	Зеленкина К. М.	ZelenkinaKM@mpei.ru
3	0020233260	Мясникова Д.А.	MysnikovaDA@mpei.ru
4			

Рис. 6. Отображение добавленного студента в таблицу stud в программе VSCode.

ЗАДАНИЕ ДЛЯ ЭТАПА 3 НИР

3.1. Цель этапа

Цель: получение первичного опыта создания программы для работы с базой данных с использованием среды Python.

3.2. Содержание этапа

1. Создать БД с одной таблицей заданной структуры.
2. Разработать программу на Python которая должна: открывать БД и показывать список полей таблицы и иметь меню с функциями:
 - Показывать таблицу на экране
 - Сохранять таблицу в текстовый файл
 - Фильтровать строки по условию (поле + логическое выражение)
 - Работать с отфильтрованными строками: удалять или изменять значения
 - Добавлять новые строки
 - Завершать работу
3. Записать данные в таблицу.

3.3. Создание таблицы, открытие файла и организация меню управления базой данных.

1. Будем использовать `if __name__ == '__main__':` для разделения двух режимов работы файла:

- в качестве библиотеки.
- в качестве самой программы.

Создадим файл с названием ***bd2.sqlite*** и переменную ***cur*** (Для выполнения всех операций с базой данных):

```
import sqlite3

if __name__ == '__main__':
    con = sqlite3.connect('bd2.sqlite')
```

```
cur = con.cursor()
main() — дальнейшая функция
```

Далее будем прописывать функции для нашей программы:

1. Данная функция создает в базе данных таблицу **educational_information** с 9 полями для хранения учебной информации, где поле **code** является уникальным первичным ключом, а остальные поля содержат данные о дисциплинах, семестрах, оценках и преподавателях.

```
def create_table(cur):
    sql = """\
CREATE TABLE IF NOT EXISTS educational_information (
    code TEXT PRIMARY KEY,
    name TEXT,
    number_semestr INTEGER,
    type_test TEXT,
    date_test TEXT,
    FIO_teacher TEXT,
    teaching_position TEXT,
    grade INTEGER,
    date_of_entry TEXT
)
"""
    cur.execute(sql)
```

2. **Функция меню** — то, что будет выводиться на экране у пользователя после открытия файла.

Данная функция сначала запрашивает у пользователя имя существующего файла БД и название таблицы, затем подключается к базе и создает таблицу при необходимости, после чего запускает основное меню с восемью опциями для управления данными — от просмотра структуры и содержимого таблицы до операций фильтрации, добавления, изменения, удаления записей и сохранения данных в файл, обеспечивая корректное закрытие соединения с БД при завершении работы.

```
def main():
    dbname = ' '
    while (os.path.isfile(dbname) != True):
        dbname = input('Укажите имя файла SQLite: ')
        if (os.path.isfile(dbname) == True):
            break
```

```
print('Нет такого файла!')
```

```
tblname = input('Укажите имя таблицы: ')
```

Подключение к БД

```
con = sqlite3.connect(dbname)  
cur = con.cursor()
```

Создание таблицы если не существует

```
create_table(cur)  
con.commit()
```

```
while True:
```

```
    print("МЕНЮ УПРАВЛЕНИЯ БАЗОЙ ДАННЫХ")  
    print("1. Показать список полей таблицы")  
    print("2. Показать все записи таблицы")  
    print("3. Сохранить таблицу в файл")  
    print("4. Фильтровать записи")  
    print("5. Удалить записи")  
    print("6. Обновить записи")  
    print("7. Добавить запись")  
    print("8. Выход")  
    print("=" * 50)
```

```
    choice = input("Выберите действие: ")
```

```
    if choice == '1':  
        display_columns(cur, tblname)
```

```
    elif choice == '2':  
        dan = select_cmd(con, cur, tblname)  
        nzap = len(dan)  
        print('\nТаблица: ', tblname, ' из БД ',
```

```
dbname)
```

```
        for i in range(nzap):  
            print(dan[i])
```

```
    elif choice == '3':  
        dan = select_cmd(con, cur, tblname)  
        filename = input("Введите имя файла для  
сохранения: ")
```



```

        save_to_file(dan, tblname, filename)

    elif choice == '4':
        filter_data(con, cur, tblname)

    elif choice == '5':
        delete_data(con, cur, tblname)

    elif choice == '6':
        update_data(con, cur, tblname)

    elif choice == '7':
        add_data(con, cur, tblname)

    elif choice == '8':
        print("Выход из программы")
        break

    else:
        print("Неверный выбор")

```

Закрытие соединения

```

cur.close()
con.close()

```

В функции **main** используются функции, которые будут описаны ниже.

3.4. Функции.

Функция выполняет *SQL-запрос* (`SELECT * FROM`) для получения всех записей из указанной таблицы базы данных, автоматически сохраняет изменения с помощью конструкции `with con` и возвращает полное содержимое таблицы в виде списка данных для дальнейшего использования.

```

def select_cmd(con, cur, tblname):
    sql = 'SELECT * FROM {}'.format(tblname)
    with con:
        data = cur.execute(sql).fetchall()
    return data

```

Функция получает и выводит на экран структуру указанной таблицы базы данных: выполняет системный запрос PRAGMA table_info для получения информации о полях таблицы, затем обрабатывает результат и выводит список всех полей с их названиями и типами данных в удобном формате.

```
def display_columns(cur, tblname):
    cur.execute(f"PRAGMA table_info({tblname})")
    columns = cur.fetchall()
    print("\nСписок полей таблицы:")
    for col in columns:
        print(f"- {col[1]} ({col[2]})")
```

Функция сохраняет данные из таблицы базы данных в текстовый файл: создает файл с указанным именем в кодировке UTF-8, записывает заголовок с названием таблицы, затем построчно сохраняет все полученные данные, а в случае ошибки выводит сообщение о проблеме.

```
def save_to_file(data, tblname, filename):
    try:
        with open(filename, 'w', encoding='utf-8') as file:
            file.write(f"Таблица: {tblname}\n")
            file.write("=" * 50 + "\n")
            for row in data:
                file.write(str(row) + "\n")
            print(f"Данные сохранены в файл: {filename}")
    except Exception as e:
        print(f"Ошибка при сохранении: {e}")
```

Функция ищет данные в таблице: пользователь вводит поле и значение, программа находит все подходящие записи и показывает их на экране, возвращая результат для дальнейшего использования.

```
def filter_data(con, cur, tblname):
    field = input("Введите имя поля для фильтрации: ")
    value = input(f"Введите значение для поля '{field}': ")

    sql = f"SELECT * FROM {tblname} WHERE {field} = ?"
    with con:
        data = cur.execute(sql, (value,)).fetchall()

    print(f"\nНайдено записей: {len(data)}")
```

```

for row in data:
    print(row)
return data

```

Функция удаляет записи из таблицы по указанному условию: пользователь выбирает поле и значение для удаления, подтверждает операцию, после чего выполняется *SQL-запрос DELETE* и выводится количество удаленных записей.

```

def delete_data(con, cur, tblname):
    field = input("Введите имя поля для условия удаления: ")
    value = input(f"Введите значение для поля '{field}': ")

    confirm = input("Вы уверены, что хотите удалить эти записи? (y/n): ")
    if confirm.lower() == 'y':
        sql = f"DELETE FROM {tblname} WHERE {field} = ?"
        with con:
            cur.execute(sql, (value,))
        print(f"Удалено записей: {cur.rowcount}")

```

Функция изменяет данные в таблице: пользователь указывает поле для поиска записей и поле для изменения с новым значением, после подтверждения выполняется *SQL-запрос UPDATE* и показывается количество обновленных записей.

```

def update_data(con, cur, tblname):
    condition_field = input("Введите имя поля для условия: ")
    condition_value = input(f"Введите значение для поля '{condition_field}': ")
    update_field = input("Введите имя поля для обновления: ")
    new_value = input(f"Введите новое значение для поля '{update_field}': ")

    confirm = input("Вы уверены, что хотите обновить записи? (y/n): ")
    if confirm.lower() == 'y':
        sql = f"UPDATE {tblname} SET {update_field} = ? WHERE {condition_field} = ?"

```

```
with con:
    cur.execute(sql, (new_value, condition_value))
print(f"Обновлено записей: {cur.rowcount}")
```

Функция добавляет новую запись в таблицу: последовательно запрашивает у пользователя все необходимые данные учебной дисциплины, формирует *SQL-запрос INSERT* с параметризацией для безопасного добавления и выполняет его, после чего подтверждает успешное добавление записи.

```
def add_data(con, cur, tblname):
    print("Введите данные для новой записи:")
    code = input("Код дисциплины: ")
    name = input("Название дисциплины: ")
    number_semestr = input("Номер семестра: ")
    type_test = input("Тип аттестации: ")
    date_test = input("Дата аттестации: ")
    FIO_teacher = input("ФИО преподавателя: ")
    teaching_position = input("Должность преподавателя: ")
    grade = input("Оценка: ")
    date_of_entry = input("Дата занесения: ")

    sql = f"""INSERT INTO {tblname}
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)"""
    with con:
        cur.execute(sql, (code, name, number_semestr,
            type_test, date_test,
                                FIO_teacher, teaching_position,
            grade, date_of_entry))
    print("Запись добавлена успешно")
```

Данные функции применяются в функции **main** (меню), описанной выше.

3.5. Работа программы.

При нажатии на кнопку «Пуск» у нас создан файл и запустилась программа:

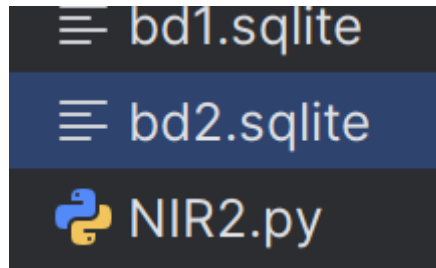


Рис. 7. Создание файла bd2.sqlite.

Для начала введём файл, который не существует:

```
Укажите имя файла SQLite: bd3.sqlite  
Нет такого файла!
```

Рис. 8. Вывод программы при вводе несуществующего файла.

Введём название существующего файла и название таблицы. Видим, что программа предлагает меню:

```
Укажите имя файла SQLite: bd2.sqlite  
Укажите имя таблицы: educational_infprmation  
МЕНЮ УПРАВЛЕНИЯ БАЗОЙ ДАННЫХ  
1. Показать список полей таблицы  
2. Показать все записи таблицы  
3. Сохранить таблицу в файл  
4. Фильтровать записи  
5. Удалить записи  
6. Обновить записи  
7. Добавить запись  
8. Выход  
=====
```

Рис. 9. Вывод меню при вводе данных, которые существуют.

Введём 1 действие. Видим, что программа показала список полей таблицы:

```
Выберите действие: 1

Список полей таблицы:
- code (TEXT)
- name (TEXT)
- number_semestr (INTEGER)
- type_test (TEXT)
- date_test (TEXT)
- FIO_teacher (TEXT)
- teaching_position (TEXT)
- grade (INTEGER)
- date_of_entry (TEXT)
```

Рис. 10. Вывод программе при вводе 1 действия.

Попробуем добавить данные в таблицу:

```
Выберите действие: 7
Введите данные для новой записи:
Код дисциплины: 45
Название дисциплины: Физика
Номер семестра: 2
Тип аттестации: экзамен
Дата аттестации: 18.06.2024
ФИО преподавателя: Корецкая И.Н.
Должность преподавателя: Профессор
Оценка: 4
Дата занесения: 20.06.2024
Запись добавлена успешно
```

Рис. 11. Внесение данных.

Теперь посмотрим эти данные (Перед этим внесла ещё данные):

```
Выберите действие: 2

Таблица: educational_information из БД bd2.sqlite
('45', 'Физика', 2, 'экзамен', '18.06.2024', 'Корецкая И.Н.', 'Профессор', 4, '20.06.2024')
('32', 'Мат. Анализ', 3, 'зачет', '23.12.2024', 'Иванов Иван Иванович', 'доцент', 5, '25.12.2024')
```

Рис. 12. Просмотр данных

Найдём данные по значению поля:

```
Выберите действие: 4
Введите имя поля для фильтрации: name
Введите значение для поля 'name': Физика

Найдено записей: 1
('45', 'Физика', 2, 'экзамен', '18.06.2024', 'Корецкая И.Н.', 'Профессор', 4, '20.06.2024')
```

Рис. 13. Нахождение данных

Попробуем теперь обновить данные. Заменим «Физику» на «Электронику»:

```
Выберите действие: 6
Введите имя поля для условия: name
Введите значение для поля 'name': Физика
Введите имя поля для обновления: name
Введите новое значение для поля 'name': Электроника
Вы уверены, что хотите обновить записи? (y/n): y
```

Рис. 14. Замена значения.

Посмотрим теперь записи в таблице. Как видно, вместо «Физики» теперь стоит «Электроника»:

```
Выберите действие: 2

Таблица: educational_information из БД bd2.sqlite
('45', 'Электроника', 2, 'экзамен', '18.06.2024', 'Корецкая И.Н.', 'Профессор', 4, '20.06.2024')
('32', 'Мат. Анализ', 3, 'зачет', '23.12.2024', 'Иванов Иван Иванович', 'доцент', 5, '25.12.2024')
```

Рис. 15. Просмотр таблицы с измененным значением.

Теперь удалим строку в таблице с данными по полю:

```
Выберите действие: 5
Введите имя поля для условия удаления: name
Введите значение для поля 'name': Электроника
Вы уверены, что хотите удалить эти записи? (y/n): y
```

Рис. 16. Процесс удаления строки по значению в поле.

Удостоверимся, что строка удалилась:

```
Выберите действие: 2

Таблица: educational_information из БД bd2.sqlite
('32', 'Мат. Анализ', 3, 'зачет', '23.12.2024', 'Иванов Иван Иванович', 'доцент', 5, '25.12.2024')
ИМЕНА УДАЛЕНИЯ БАЗЫ ДАННЫХ
```

Рис. 17. Оставшиеся данные, после удаления строки по значению в поле.

Выйдем из программы:

```
Выберите действие: 8
Выход из программы
```

Рис. 18. Выход из программы.

3.6. Полноценный код программы.

```
import sqlite3
import os

def create_table(cur):
    sql = """\
CREATE TABLE IF NOT EXISTS educational_information (
    code TEXT PRIMARY KEY,
    name TEXT,
    number_semestr INTEGER,
    type_test TEXT,
    date_test TEXT,
    FIO_teacher TEXT,
    teaching_position TEXT,
    grade INTEGER,
    date_of_entry TEXT
)
"""
    cur.execute(sql)
```



```

def select_cmd(con, cur, tblname):
    sql = 'SELECT * FROM {}'.format(tblname)
    with con:
        data = cur.execute(sql).fetchall()
    return data

def display_columns(cur, tblname):
    cur.execute(f"PRAGMA table_info({tblname})")
    columns = cur.fetchall()
    print("\nСписок полей таблицы:")
    for col in columns:
        print(f"- {col[1]} ({col[2]})")

def save_to_file(data, tblname, filename):
    try:
        with open(filename, 'w', encoding='utf-8') as file:
            file.write(f"Таблица: {tblname}\n")
            file.write("=" * 50 + "\n")
            for row in data:
                file.write(str(row) + "\n")
            print(f"Данные сохранены в файл: {filename}")
    except Exception as e:
        print(f"Ошибка при сохранении: {e}")

def filter_data(con, cur, tblname):
    field = input("Введите имя поля для фильтрации: ")
    value = input(f"Введите значение для поля '{field}': ")

    sql = f"SELECT * FROM {tblname} WHERE {field} = ?"
    with con:
        data = cur.execute(sql, (value,)).fetchall()

    print(f"\nНайдено записей: {len(data)}")
    for row in data:
        print(row)
    return data

```

```

def delete_data(con, cur, tblname):
    field = input("Введите имя поля для условия удаления: ")
    value = input(f"Введите значение для поля '{field}': ")

    confirm = input("Вы уверены, что хотите удалить эти записи? (y/n): ")
    if confirm.lower() == 'y':
        sql = f"DELETE FROM {tblname} WHERE {field} = ?"
        with con:
            cur.execute(sql, (value,))
            print(f"Удалено записей: {cur.rowcount}")

def update_data(con, cur, tblname):
    condition_field = input("Введите имя поля для условия: ")
    condition_value = input(f"Введите значение для поля '{condition_field}': ")
    update_field = input("Введите имя поля для обновления: ")
    new_value = input(f"Введите новое значение для поля '{update_field}': ")

    confirm = input("Вы уверены, что хотите обновить записи? (y/n): ")
    if confirm.lower() == 'y':
        sql = f"UPDATE {tblname} SET {update_field} = ? WHERE {condition_field} = ?"
        with con:
            cur.execute(sql, (new_value, condition_value))
            print(f"Обновлено записей: {cur.rowcount}")

def add_data(con, cur, tblname):
    print("Введите данные для новой записи:")
    code = input("Код дисциплины: ")
    name = input("Название дисциплины: ")
    number_semestr = input("Номер семестра: ")
    type_test = input("Тип аттестации: ")
    date_test = input("Дата аттестации: ")

```

```

FIO_teacher = input("ФИО преподавателя: ")
teaching_position = input("Должность преподавателя: ")
grade = input("Оценка: ")
date_of_entry = input("Дата занесения: ")

sql = f"""INSERT INTO {tblname}
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)"""
with con:
    cur.execute(sql, (code, name, number_semestr,
type_test, date_test,
                                FIO_teacher, teaching_position,
grade, date_of_entry))
    print("Запись добавлена успешно")

def main():
    dbname = ' '
    while (os.path.isfile(dbname) != True):
        dbname = input('Укажите имя файла SQLite: ')
        if (os.path.isfile(dbname) == True):
            break
        print('Нет такого файла!')

tblname = input('Укажите имя таблицы: ')

# Подключение к БД
con = sqlite3.connect(dbname)
cur = con.cursor()

# Создание таблицы если не существует
create_table(cur)
con.commit()

while True:
    print("МЕНЮ УПРАВЛЕНИЯ БАЗОЙ ДАННЫХ")
    print("1. Показать список полей таблицы")
    print("2. Показать все записи таблицы")
    print("3. Сохранить таблицу в файл")
    print("4. Фильтровать записи")
    print("5. Удалить записи")

```

```
print("6. Обновить записи")
print("7. Добавить запись")
print("8. Выход")
print("=" * 50)
```

```
choice = input("Выберите действие: ")
```

```
if choice == '1':
    display_columns(cur, tblname)
```

```
elif choice == '2':
    dan = select_cmd(con, cur, tblname)
    nzap = len(dan)
    print('\nТаблица: ', tblname, ' из БД ', dbname)
    for i in range(nzap):
        print(dan[i])
```

```
elif choice == '3':
    dan = select_cmd(con, cur, tblname)
    filename = input("Введите имя файла для сохранения: ")
    save_to_file(dan, tblname, filename)
```

```
elif choice == '4':
    filter_data(con, cur, tblname)
```

```
elif choice == '5':
    delete_data(con, cur, tblname)
```

```
elif choice == '6':
    update_data(con, cur, tblname)
```

```
elif choice == '7':
    add_data(con, cur, tblname)
```

```
elif choice == '8':
    print("Выход из программы")
    break
```

```
else:
```

```
print("Неверный выбор")

# Закрытие соединения
cur.close()
con.close()

if __name__ == '__main__':
    con = sqlite3.connect('bd2.sqlite')
    cur = con.cursor()
    main()
```

ЗАДАНИЕ ДЛЯ ЭТАПА 4 НИР

(Вариант 26)

4.1. Цель этапа

Цель: Разработать программу на Python для анализа данных о вузах России из базы данных Vuz.sqlite, реализующую два задания:

1. Фильтрацию вузов по проценту преподавателей с учеными степенями
2. Статистику распределения ведущих вузов (ФУ/НИУ) по федеральным округам

4.2. Структура БД

Исходные данные для выполнения индивидуальных заданий содержатся в БД **Vuz.sqlite**. При выполнении заданий будут использоваться данные следующих таблиц.

1. Таблица «Картотека вузов» – vuzkart с записями, содержащими сведения о вузах России.

Ключ	Имя поля	Тип данных	Описание
V	Codvuz	char(6)	Код вуза
	z1	char(200)	Полное наименование вуза
	z1full	char(250)	Полное юридическое наименование вуза
	z2	char(12)	Сокращённое наименование
	z2ustav	char(100)	Сокращённое наименование по уставу
	foundyear	integer(4)	Год основания вуза
	z8	char(110)	Юридический адрес вуза
	z9	char(40)	Справочный телефон вуза
	z12	char(20)	Телефон руководителя вуза
	e_mail	char(30)	Адрес электронной почты
	www	char(30)	Адрес официального сайта вуза
	z15	char(40)	Ректор вуза
	region	char(17)	Федеральный округ
	city	char(20)	Город
	status	char(15)	Статус вуза: университет, академия, институт
	oblname	char(40)	Субъект федерации по расположению вуза
	gr_ved	char(3)	Категория вуза – принадлежность к ведущим вузам (ФУ – федеральный университет, НИУ – национальный исследовательский университет)
	prof	char(2)	Профиль вуза: ИТ – инженерно-технический, КЛ – классический университет, ГП – гуманитарно-педагогический, МП – многопрофильный

Рис. 19. Структура таблицы *vuzkart*.

	codvuz	z1	z1full	z2	z2ustav
	Filter...	Filter...	Filter...	Filter...	Filter...
1	1	Алтайский государственный технический университе...	Федеральное государственное бюджетное образоват...	АлтГТУ	АлтГТУ
2	2	Алтайский государственный университет	Федеральное государственное бюджетное образоват...	АлтГУ	ФГБОУ ВПО "Алтай
4	4	Северный (Арктический) федеральный университет и...	Федеральное государственное автономное образова...	Сев(Аркт)ФУ	САФУ
		Балтийский государственный технический университ...	Федеральное государственное бюджетное образоват...	БалтГТУ	БГТУ "ВОЕНМЕХ" и
5	9	Амурский государственный университет	Федеральное государственное бюджетное образоват...	АмурГУ	ФГБОУ ВПО "АмГУ"
6	11	Брянский государственный технический университет	Федеральное государственное бюджетное образоват...	БрянГТУ	ФГБОУ ВПО "БГТУ"
7	13	Владимирский государственный университет имени ...	Федеральное государственное бюджетное образоват...	ВладГУ	ВлГУ
8	19	Воронежский государственный архитектурно-строит...	Федеральное государственное бюджетное образоват...	ВорГАСУ	Воронежский ГАСУ
9	28	Российский государственный университет нефти и га...	Федеральное государственное бюджетное образоват...	РГУНГ	ФГБОУ ВПО "РГУ н
10	29	Государственный университет управления	Федеральное государственное бюджетное образоват...	ГУУ	ГУУ
11	33	Дагестанский государственный университет	Федеральное государственное бюджетное образоват...	ДагГУ	ДГУ
12	34	Дагестанский государственный технический универс...	Федеральное государственное бюджетное образоват...	ДагГТУ	ФГБОУ ВПО "ДГТУ"
13	36	Дальневосточный федеральный университет	Федеральное государственное автономное образова...	ДВФУ	ДВФУ
14	40	Ивановский государственный университет	Федеральное государственное бюджетное образоват...	ИвГУ	ИвГУ

Рис. 20. Первые 4 столбца таблицы *vuzkart* в *Visual Studio Code*.

2. Таблица vuzstat, содержащая статистические данные по вузам.

Ключ	Имя поля	Тип данных	Описание
V	codvuz	char(6)	Код вуза (вторичный)
	PPS	integer(4)	Численность профессорско-преподавательского состава (ППС)
	PR	integer(4)	ППС – профессеры
	DC	integer(4)	ППС – доценты
	DN	integer(4)	ППС – доктора наук
	KN	integer(4)	ППС – кандидаты наук
	ZOB	integer(4)	Количество зданий общежитий
	STUD	integer(4)	Количество студентов, всего
	BAC	integer(4)	Обучается по программам бакалавриата
	SPEC	integer(4)	Обучается по программам подготовки специалистов
	MAG	integer(4)	Обучается по программам магистратуры
	ST_OCH	integer(4)	Студентов по очной форме обучения
	ST_OCH_Z	integer(4)	Студентов по очно-заочной форме обучения
	ST_Z	integer(4)	Студентов по заочной форме обучения
	ASP	integer(4)	Аспирантов всего
	ASP_OCH	integer(4)	Аспирантов по очной форме обучения

Рис. 21. Структура таблицы *vuzstat*.

pps	#	pr	#	dc	#	dn	#	kn	#	zob	#	stud	#	bac	#	spec	#
Filter		Filter		Filter		Filter		Filter		Filter		Filter		Filter		Filter	
	1233		136		681		107		674		7		18939		3550		15
	854		63		411		92		479		3		15547		4724		9
	1197		80		504		91		611		13		18550		4065		14
	528		102		278		66		356		3		5296		792		4
	592		53		260		44		279		5		9223		2228		6
	402		30		214		33		237		5		8034		1598		6
	1287		131		596		136		674		13		25680		6053		18
	569		80		278		60		291		6		9932		2190		7
	846		181		366		190		409		5		10543		3198		6
	1063		194		560		182		635		2		15073		2101		12
	1204		128		552		144		754		4		18799		3915		14
	747		108		217		110		443		3		9991		2196		7
	2398		321		1393		241		1660		14		53636		6813		45

Рис. 22. Первые 8 столбцов таблицы *vuzstat* в Visual Studio Code .

4.2. Описание алгоритма по 1 заданию.

Задание 1

1. Обеспечить выбор порогового значения процента имеющих ученые степени преподавателей вузов, интересующего пользователя (при этом должны быть подсказки по мин. и макс. значениям этой величины). Составить и отобразить на экране перечень полных наименований вузов, в которых значение этого процента не превышает выбранного порога.

Описание алгоритма:

1. Программа вычисляет диапазон значений процента преподавателей с учеными степенями по всем вузам по формуле:

$$\text{Процент} = \frac{\text{Кандидат наук} + \text{Доктор наук}}{\text{Общее число преподавателей}} \times 100\%$$

2. Показывает пользователю минимальное и максимальное значения
3. Запрашивает у пользователя пороговое значение процента
4. Это значение определяет, какие вузы будут включены в результаты
5. Находит вузы, где процент преподавателей с учеными степенями меньше или равен заданному значению
6. Объединяет данные из таблиц названий вузов и статистики преподавателей
7. Формирует нумерованный список найденных вузов в алфавитном порядке
8. Указывает общее количество найденных вузов
9. Сообщает, если подходящих вузов не обнаружено

Рассмотрим как данный алгоритм реализован в коде:

Код:

```
def task1_vuz(con, cur):  
    """Задача 1: Вузы с процентом преподавателей с учеными  
    степенями <= порога"""  
    print("Процент преподавателей с учеными степенями")
```

Программа находит минимальное и максимальное значение процента по всем вузам:

```
sql = """  
SELECT  
    MIN((CAST(vuzstat.KN + vuzstat.DN AS FLOAT) /  
vuzstat.PPS) * 100) as min_percent,  
    MAX((CAST(vuzstat.KN + vuzstat.DN AS FLOAT) /  
vuzstat.PPS) * 100) as max_percent
```



```
FROM vuzstat
WHERE vuzstat.PPS > 0
"""
```

Выполняет SQL-запрос и получает одну строку с двумя числами: мин. процент и макс. процент.

```
try:
    with con:
        result = cur.execute(sql).fetchone()
```

Убеждается, что в базе есть данные для расчета. Если данных нет - программа останавливается.

```
if not result or result[0] is None:
    print("Нет данных для расчета процента
преподавателей с учеными степенями.")
    return
```

Разделяет результат на два отдельных числа и выводит информацию пользователю.

```
min_percent, max_percent = result
print(f"Минимальный процент преподавателей с учеными
степенями: {min_percent:.2f}%")
print(f"Максимальный процент преподавателей с учеными
степенями: {max_percent:.2f}%")
```

Запрашиваем у пользователя пороговое значение и пользователь вводит границу - какой максимальный процент его интересует.

```
try:
    threshold = float(
        input(f"\nВведите пороговое значение процента
(от {min_percent:.2f} до {max_percent:.2f}): "))
```

Если пользователь вводит некорректные данные, то программа сообщает об ошибке.

```
except ValueError:
    print("Ошибка: введите число.")
```

```
return
```

Выполняется поиск вузов, где процент не превышает заданный порог:

```
sql = """  
SELECT vuzkart.z1full
```

Соединяем две таблицы:

```
FROM vuzkart  
JOIN vuzstat ON vuzkart.Codvuz = vuzstat.codvuz
```

Проверяем условия:

```
WHERE vuzstat.PPS > 0  
      AND (CAST(vuzstat.KN + vuzstat.DN AS FLOAT) /  
vuzstat.PPS) * 100 <= ?  
ORDER BY vuzkart.z1full  
"""
```

Выполняет поиск вузов по условию пользователя и сохраняет результат.

```
with con:  
    data = cur.execute(sql, (threshold,)).fetchall()
```

Вывод результата:

```
print(f"\nВузы, где процент преподавателей с учеными  
степенями <= {threshold:.2f}%:")  
if data:  
    for i, row in enumerate(data, 1):  
        print(f"{i:3}. {row[0]}")  
    print(f"Всего найдено: {len(data)}")
```

Если программа не нашла вузы, удовлетворяющие нашему условию:

```
else:  
    print("Вузов, удовлетворяющих условию, не  
найдено.")
```

```
except sqlite3.Error as e:  
    print(f"Ошибка базы данных: {e}")  
    print("Убедитесь, что база данных содержит таблицы  
vuzkart и vuzstat.")
```

4.3. Описание алгоритма по 2 заданию.

Задание 2

2. Обеспечить возможность пользователю выбрать из списка по принадлежности к группе ведущих вузов (ФУ, НИУ, ФУ и НИУ). Для вузов выбранной группы рассчитать и представить в виде таблицы распределение ведущих вузов выбранной группы по федеральным округам. Таблица должна иметь поля: порядковый номер, федеральный округ, количество ведущих вузов выбранной группы в данном федеральном округе, процент от общего числа ведущих вузов в выбранной группе. Таблица должна быть упорядочена по федеральным округам. Последняя строка таблицы – итоговая, со значениями: в поле «Федеральный округ» – значение «Все», в поле «Количество ведущих вузов» – общее число ведущих вузов выбранной группы.

Описание алгоритма:

1. Программа предоставляет пользователю интерактивное меню для выбора категории анализируемых вузов. Реализованы три варианта:
 - Федеральные университеты (ФУ)
 - Национальные исследовательские университеты (НИУ)
 - Объединенная выборка (ФУ и НИУ)
2. На основе выбора формируется соответствующий SQL-фильтр и текстовая метка для последующего отображения.
3. Выполняется SQL-запрос к базе данных, который осуществляет:
 - Выборку наименований федеральных округов
 - Подсчет количества вузов выбранной категории в каждом округе
 - Фильтрацию записей с корректно указанными округами
 - Группировку результатов по территориальному признаку
 - Сортировку в алфавитном порядке
3. Производится проверка наличия данных. При отсутствии вузов выбранной категории выполнение алгоритма прекращается.
4. Создается структурированная таблица, содержащая:
 - . Заголовок с наименованиями колонок
 - . Тело таблицы с данными по каждому федеральному округу:
 - а. Порядковый номер
5. Процентное соотношение (расчет: $(\text{количество в округе} / \text{общее количество}) \times 100\%$)

6 Итоговую строку с суммарными значениями.

Код:

```
def task2_vuz(con, cur):  
    """Задача 2: Распределение ведущих вузов по федеральным  
    округам"""  
    print("Распределение ведущих вузов по федеральным  
    округам:")
```

Спрашиваем у пользователя, какие вузы анализировать.

```
    print("Выберите группу ведущих вузов:")  
    print("1. Федеральные университеты (ФУ)")  
    print("2. Национальные исследовательские университеты  
(НИУ)")  
    print("3. ФУ и НИУ")  
    choice = input("Ваш выбор (1/2/3): ")
```

Создаём условия для SQL-запроса в зависимости от выбора.

```
    if choice == '1':  
        group_filter = "gr_ved = 'ФУ'"  
        group_label = "Федеральные университеты"  
    elif choice == '2':  
        group_filter = "gr_ved = 'НИУ'"  
        group_label = "Национальные исследовательские  
университеты"  
    elif choice == '3':  
        group_filter = "gr_ved IN ('ФУ', 'НИУ')"  
        group_label = "ФУ и НИУ"  
    else:  
        print("Неверный выбор.")  
        return
```

SQL-запрос выполняет выборку наименований федеральных округов с подсчетом количества вузов для каждой выбранной категории (ФУ/НИУ), исключая записи с неуказанным округом, группируя результаты по округам и сортируя их в алфавитном порядке.

```
    try:
```

```

sql = f"""
SELECT
    region,
    COUNT(*) as count
FROM vuzkart
WHERE {group_filter} AND region IS NOT NULL AND region
!= ''

GROUP BY region
ORDER BY region
"""

```

Программа выполняет запрос и получает все данные, затем проверяет, есть ли результаты. Если нет, то сообщает об этом.

```

with con:
    data = cur.execute(sql).fetchall()

    if not data:
        print(f"В выбранной группе ({group_label}) нет
вузов.")
    return

```

Считаем общее количество вузов выбранной категории.

```
total = sum(row[1] for row in data)
```

Программа формирует заголовок таблицы с выровненными колонками:

```

print(f"\nРаспределение {group_label} по федеральным
округам:")
print(f"{'№':<3} {'Федеральный округ':<35} {'Кол-
во':<8} {'%':<6}")

```

Для каждого федерального округа программа присваивает порядковый номер, вычисляет процентное соотношение путем деления количества вузов в округе на общее количество и умножения на 100%, после чего форматирует и выводит строку с выравниванием: номер выравнивается влево на 3 символа, название округа - влево на

35 символов, количество вузов - влево на 8 символов, а процент отображается как число с двумя знаками после запятой.

```
for i, (region, count) in enumerate(data, 1):
    percent = (count / total) * 100 if total > 0 else 0
    print(f"{i:<3} {region:<35} {count:<8}
{percent:5.2f}%")
```

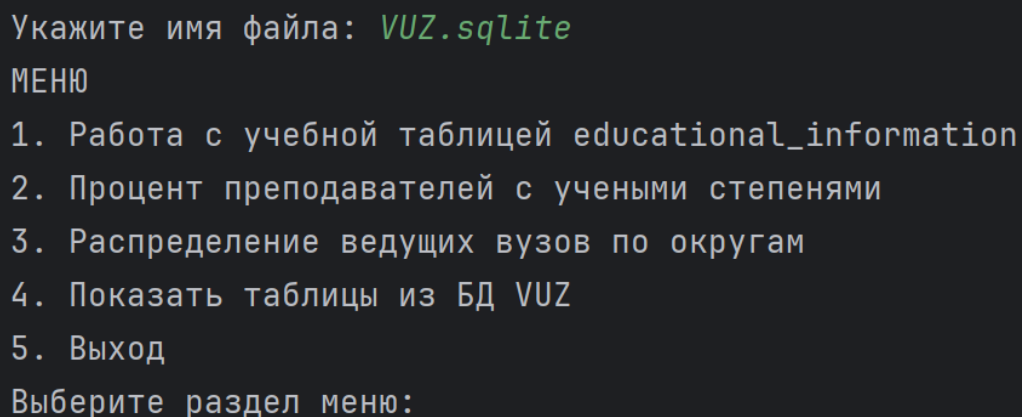
Добавление итоговой строки:

```
print("-" * 70)
print(f"{'Всего':<3} {'Всё':<35} {total:<8}
{100.00:5.2f}%")
```

```
except sqlite3.Error as e:
    print(f"Ошибка базы данных: {e}")
```

4.4. Результаты тестирования.

Запустим код и укажем имя нужного нам файла. Программа вывела меню.



```
Укажите имя файла: VUZ.sqlite
МЕНЮ
1. Работа с учебной таблицей educational_information
2. Процент преподавателей с учеными степенями
3. Распределение ведущих вузов по округам
4. Показать таблицы из БД VUZ
5. Выход
Выберите раздел меню:
```

Рис. 23. Запуск кода и ввод имя файла.

Выберем раздел 2, так как раздел 1 уже тестировали на 3 Этапе. Введём минимальный процент.

Выберите раздел меню: 2
Процент преподавателей с учеными степенями
Минимальный процент преподавателей с учеными степенями: 32.36%
Максимальный процент преподавателей с учеными степенями: 88.15%
Введите пороговое значение процента (от 32.36 до 88.15): 32.36
Вузы, где процент преподавателей с учеными степенями $\leq 32.36\%$:
Вузов, удовлетворяющих условию, не найдено.

Рис. 24. Ввод минимального порогового значения.

Программа показала диапазон: минимальный процент в вузах = 32.36%, максимальный = 88.15%. Мы ввели значение 32.36. Программа не нашла ни одного вуза - значит, нет вузов с процентом ниже или равным 32.36%

Введём пороговое значение 67.98.

Выберите раздел меню: 2
Процент преподавателей с учеными степенями
Минимальный процент преподавателей с учеными степенями: 32.36%
Максимальный процент преподавателей с учеными степенями: 88.15%
Введите пороговое значение процента (от 32.36 до 88.15): 67.98

Рис. 25. Ввод порогового значения, равного 67.98.

Вузы, где процент преподавателей с учеными степенями $\leq 67.98\%$:

1. Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования "Российский государственный университет туризма и се
2. Федеральное государственное автономное образовательное учреждение высшего профессионального образования "Балтийский федеральный университет имени Иммануила
3. Федеральное государственное автономное образовательное учреждение высшего профессионального образования "Дальневосточный федеральный университет"
4. Федеральное государственное автономное образовательное учреждение высшего профессионального образования "Московский физико-технический институт (государств
5. Федеральное государственное автономное образовательное учреждение высшего профессионального образования "Национальный исследовательский ядерный университет
6. Федеральное государственное автономное образовательное учреждение высшего профессионального образования "Северный (Арктический) федеральный университет имен
7. Федеральное государственное автономное образовательное учреждение высшего профессионального образования "Северо-Восточный федеральный университет имени М.К.
8. Федеральное государственное автономное образовательное учреждение высшего профессионального образования "Сибирский федеральный университет"
9. Федеральное государственное автономное образовательное учреждение высшего профессионального образования "Уральский федеральный университет имени первого Пре
10. Федеральное государственное автономное образовательное учреждение высшего профессионального образования "Южный федеральный университет"
11. Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования "Московский государственный университет геодезии и к
12. Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования "Алтайский государственный технический университет им
13. Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования "Алтайский государственный университет"
14. Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования "Амурский государственный университет"

Рис. 26. Вывод программы.

```
75. Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования "Чеченский госуда
76. Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования "Чеченский госуда
77. Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования "Чувашский госуда
78. Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования "Южно-Уральский г
Всего найдено: 78
```

Рис. 27. Вывод программы.

Программа нашла 78 вузов, где процент преподавателей с учеными степенями не превышает 67.98%.

Выберите раздел меню: 2

Процент преподавателей с учеными степенями

Минимальный процент преподавателей с учеными степенями: 32.36%

Максимальный процент преподавателей с учеными степенями: 88.15%

Введите пороговое значение процента (от 32.36 до 88.15): 88.15

Вузы, где процент преподавателей с учеными степенями $\leq$ 88.15%:

1. Федеральное государственное бюджетное образовательное учреждение высшего профессионального обр
2. Федеральное государственное автономное образовательное учреждение высшего профессионального обр
3. Федеральное государственное автономное образовательное учреждение высшего профессионального обр
4. Федеральное государственное автономное образовательное учреждение высшего профессионального обр
5. Федеральное государственное автономное образовательное учреждение высшего профессионального обр

```
117. Федеральное государственное бюджетное образовательное учреждение высшего профессионального
118. Федеральное государственное бюджетное образовательное учреждение высшего профессионального
119. Федеральное государственное бюджетное образовательное учреждение высшего профессионального
120. Федеральное государственное бюджетное образовательное учреждение высшего профессионального
Всего найдено: 120
```

Рис. 27. Вывод программы при вводе максимального значения.

Программа нашла 120 вузов, где процент преподавателей с учеными степенями $\leq$ 88.15%.

Выберем раздел 3. Программа предложила выбрать категорию вузов.

Выберите раздел меню: 3

Распределение ведущих вузов по федеральным округам:

Выберите группу ведущих вузов:

1. Федеральные университеты (ФУ)
2. Национальные исследовательские университеты (НИУ)
3. ФУ и НИУ

Ваш выбор (1/2/3):

Рис. 28. Вывод программы при выборе раздела 3.

Выберем 1 вариант. Видим, что база данных не содержит информации о федеральных университетах.

Выберите группу ведущих вузов:

1. Федеральные университеты (ФУ)
2. Национальные исследовательские университеты (НИУ)
3. ФУ и НИУ

Ваш выбор (1/2/3): 1

В выбранной группе (Федеральные университеты) нет вузов.

Рис. 29. Вывод программы при выборе 1 варианта (ФУ).

Ваш выбор (1/2/3): 2

Распределение Национальные исследовательские университеты по федеральным округам:

№	Федеральный округ	Кол-во	%
1	Приволжский	8	30.77%
2	Северо-Западный	3	11.54%
3	Сибирский	4	15.38%
4	Уральский	1	3.85%
5	Центральный	10	38.46%

Всего Все		26	100.00%

Рис. 30. Вывод программы при выборе 2 варианта (НИУ).

В базе найдено 26 национальных исследовательских университетов (НИУ). НИУ сконцентрированы в центральных регионах России - почти 70% находятся в Центральном и Приволжском округах.

Затем выберем вариант 3.

Ваш выбор (1/2/3): 3

Распределение ФУ и НИУ по федеральным округам:

№	Федеральный округ	Кол-во	%
1	Приволжский	8	30.77%
2	Северо-Западный	3	11.54%
3	Сибирский	4	15.38%
4	Уральский	1	3.85%
5	Центральный	10	38.46%

Всего Все		26	100.00%

Рис. 31. Вывод программы при выборе 3 варианта.

Всего 26 ведущих вузов в России (только НИУ, ФУ не найдены). Почти 70% всех ведущих вузов России находятся всего в двух регионах - Центральном и Приволжском округах. Половина страны (3 из 8 округов) вообще не имеет ведущих университетов.

Чтобы посмотреть содержимое БД VUZ выберем пункт 4.

```

Выберите раздел меню: 4
ПРОСМОТР ТАБЛИЦ БАЗЫ ДАННЫХ

Доступные таблицы:
1. educational_information
2. vuz_rf
3. vuzkart
4. vuzstat

```

Рис. 32. Вывод программы при выборе раздела 4.

В базе данных обнаружено 4 таблицы: учебная таблица educational_information, таблица vuz_rf, таблица vuzkart и таблица vuzstat.

Посмотрим таблицу 3.

```
Выберите номер таблицы (1-4) или введите 0 для возврата: 3

Структура таблицы 'vuzkart':
  codvuz (TEXT)
  z1 (TEXT)
  z1full (TEXT)
  z2 (TEXT)
  z2ustav (TEXT)
  foundyear (INTEGER)
  z8 (TEXT)
  z9 (TEXT)
  z12 (TEXT)
  e_mail (TEXT)
  www (TEXT)
  z15 (TEXT)
  region (TEXT)
  city (TEXT)
  status (TEXT)
  oblname (TEXT)
  gr_ved (TEXT)
  prof (TEXT)

Введите число (0 - все):
```

Рис. 33. Состав таблицы 3.

Посмотрим её содержимое. Выведем первые 5 строк.

```
Введите число (0 - все): 5
Показаны первые 5 строк

Первые 5 записей из таблицы: vuzkart (всего записей: 120)
=====
(' 1', 'Алтайский государственный технический университет имени И.И. Ползунова
(' 2', 'Алтайский государственный университет
(' 4', 'Северный (Арктический) федеральный университет имени М.В. Ломоносова
(' 6', 'Балтийский государственный технический университет "ВОЕНМЕХ" им. Д.Ф. Устинова
(' 9', 'Амурский государственный университет
```

Рис. 34. Содержимое таблицы 3.

Выход из программы:

```
МЕНЮ
1. Работа с учебной таблицей educational_information
2. Процент преподавателей с учеными степенями
3. Распределение ведущих вузов по округам
4. Показать таблицы из БД VUZ
5. Выход
Выберите раздел меню: 5
Выход из программы
```

Рис. 35. Выход из программы.

4.5. Полноценный код программы.

```
import sqlite3
import os

def create_table(cur):
    sql = """\
CREATE TABLE IF NOT EXISTS educational_information (
    code TEXT PRIMARY KEY,
    name TEXT,
    number_semestr INTEGER,
    type_test TEXT,
    date_test TEXT,
    FIO_teacher TEXT,
    teaching_position TEXT,
    grade INTEGER,
    date_of_entry TEXT
)
"""
    cur.execute(sql)

def select_cmd(con, cur, tblname):
    sql = 'SELECT * FROM {}'.format(tblname)
    with con:
        data = cur.execute(sql).fetchall()
    return data
```

```

def display_columns(cur, tblname):
    cur.execute(f"PRAGMA table_info({tblname})")
    columns = cur.fetchall()
    print("\nСписок полей таблицы:")
    for col in columns:
        print(f"- {col[1]} ({col[2]})")

def save_to_file(data, tblname, filename):
    try:
        with open(filename, 'w', encoding='utf-8') as file:
            file.write(f"Таблица: {tblname}\n")
            file.write("=" * 50 + "\n")
            for row in data:
                file.write(str(row) + "\n")
            print(f"Данные сохранены в файл: {filename}")
    except Exception as e:
        print(f"Ошибка при сохранении: {e}")

def filter_data(con, cur, tblname):
    field = input("Введите имя поля для фильтрации: ")
    value = input(f"Введите значение для поля '{field}': ")

    sql = f"SELECT * FROM {tblname} WHERE {field} = ?"
    with con:
        data = cur.execute(sql, (value,)).fetchall()

    print(f"\nНайдено записей: {len(data)}")
    for row in data:
        print(row)
    return data

def delete_data(con, cur, tblname):
    field = input("Введите имя поля для условия удаления: ")
    value = input(f"Введите значение для поля '{field}': ")

```

```

    confirm = input("Вы уверены, что хотите удалить эти записи?
(y/n): ")
    if confirm.lower() == 'y':
        sql = f"DELETE FROM {tblname} WHERE {field} = ?"
        with con:
            cur.execute(sql, (value,))
            print(f"Удалено записей: {cur.rowcount}")

```

```

def update_data(con, cur, tblname):
    condition_field = input("Введите имя поля для условия: ")
    condition_value = input(f"Введите значение для поля
'{condition_field}': ")
    update_field = input("Введите имя поля для обновления: ")
    new_value = input(f"Введите новое значение для поля
'{update_field}': ")

```

```

    confirm = input("Вы уверены, что хотите обновить записи?
(y/n): ")
    if confirm.lower() == 'y':
        sql = f"UPDATE {tblname} SET {update_field} = ? WHERE
{condition_field} = ?"
        with con:
            cur.execute(sql, (new_value, condition_value))
            print(f"Обновлено записей: {cur.rowcount}")

```

```

def add_data(con, cur, tblname):
    print("Введите данные для новой записи:")
    code = input("Код дисциплины: ")
    name = input("Название дисциплины: ")
    number_semestr = input("Номер семестра: ")
    type_test = input("Тип аттестации: ")
    date_test = input("Дата аттестации: ")
    FIO_teacher = input("ФИО преподавателя: ")
    teaching_position = input("Должность преподавателя: ")
    grade = input("Оценка: ")
    date_of_entry = input("Дата занесения: ")

    sql = f""""INSERT INTO {tblname}

```

```

VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)"""
with con:
    cur.execute(sql, (code, name, number_semestr,
type_test, date_test,
FIO_teacher, teaching_position,
grade, date_of_entry))
    print("Запись добавлена успешно")

def task1_vuz(con, cur):
    """Задача 1: Вузы с процентом преподавателей с учеными
степенями <= порога"""
    print("Процент преподавателей с учеными степенями")

    # Находим мин. и макс. процент преподавателей с учеными
степенями
    sql = """
SELECT
    MIN((CAST(vuzstat.KN + vuzstat.DN AS FLOAT) /
vuzstat.PPS) * 100) as min_percent,
    MAX((CAST(vuzstat.KN + vuzstat.DN AS FLOAT) /
vuzstat.PPS) * 100) as max_percent
FROM vuzstat
WHERE vuzstat.PPS > 0
    """

    try:
        with con:
            result = cur.execute(sql).fetchone()

            if not result or result[0] is None:
                print("Нет данных для расчета процента
преподавателей с учеными степенями.")
                return

            min_percent, max_percent = result
            print(f"Минимальный процент преподавателей с учеными
степенями: {min_percent:.2f}%")
            print(f"Максимальный процент преподавателей с учеными
степенями: {max_percent:.2f}%")

```

```

try:
    threshold = float(
        input(f"\nВведите пороговое значение процента
(от {min_percent:.2f} до {max_percent:.2f}): ")
    )
except ValueError:
    print("Ошибка: введите число.")
    return

# Запрос для получения списка вузов
sql = """
SELECT vuzkart.z1full
FROM vuzkart
JOIN vuzstat ON vuzkart.Codvuz = vuzstat.codvuz
WHERE vuzstat.PPS > 0
      AND (CAST(vuzstat.KN + vuzstat.DN AS FLOAT) /
vuzstat.PPS) * 100 <= ?
ORDER BY vuzkart.z1full
"""

with con:
    data = cur.execute(sql, (threshold,)).fetchall()

    print(f"\nВузы, где процент преподавателей с учеными
степенями <= {threshold:.2f}%:")
    if data:
        for i, row in enumerate(data, 1):
            print(f"{i:3}. {row[0]}")
        print(f"Всего найдено: {len(data)}")
    else:
        print("Вузов, удовлетворяющих условию, не
найдено.")

except sqlite3.Error as e:
    print(f"Ошибка базы данных: {e}")
    print("Убедитесь, что база данных содержит таблицы
vuzkart и vuzstat.")

def task2_vuz(con, cur):
    """Задача 2: Распределение ведущих вузов по федеральным

```


округам"""

```
print("Распределение ведущих вузов по федеральным  
округам:")
```

```
print("Выберите группу ведущих вузов:")  
print("1. Федеральные университеты (ФУ)")  
print("2. Национальные исследовательские университеты  
(НИУ)")  
print("3. ФУ и НИУ")  
choice = input("Ваш выбор (1/2/3): ")
```

```
if choice == '1':  
    group_filter = "gr_ved = 'ФУ'"  
    group_label = "Федеральные университеты"  
elif choice == '2':  
    group_filter = "gr_ved = 'НИУ'"  
    group_label = "Национальные исследовательские  
университеты"  
elif choice == '3':  
    group_filter = "gr_ved IN ('ФУ', 'НИУ')"  
    group_label = "ФУ и НИУ"  
else:  
    print("Неверный выбор.")  
    return
```

```
try:  
    # Запрос для подсчета по федеральным округам  
    sql = f"""  
    SELECT  
        region,  
        COUNT(*) as count  
    FROM vuzkart  
    WHERE {group_filter} AND region IS NOT NULL AND region  
!= ''  
    GROUP BY region  
    ORDER BY region  
    """  
  
    with con:  
        data = cur.execute(sql).fetchall()
```

```

        if not data:
            print(f"В выбранной группе ({group_label}) нет вузов.")
            return

        total = sum(row[1] for row in data)

        print(f"\nРаспределение {group_label} по федеральным округам:")
        print(f"{'№':<3} {'Федеральный округ':<35} {'Кол-во':<8} {'%':<6}")
        for i, (region, count) in enumerate(data, 1):
            percent = (count / total) * 100 if total > 0 else 0
            print(f"{i:<3} {region:<35} {count:<8} {percent:5.2f}%")

        print("-" * 70)
        print(f"{'Всего':<3} {'Всё':<35} {total:<8} {100.00:5.2f}%")

    except sqlite3.Error as e:
        print(f"Ошибка базы данных: {e}")

def show_any_table(con, cur):
    """Показать любую таблицу из базы данных"""
    print("ПРОСМОТР ТАБЛИЦ БАЗЫ ДАННЫХ")

    try:
        # Получаем список всех таблиц
        sql = "SELECT name FROM sqlite_master WHERE type='table' ORDER BY name"
        with con:
            tables = cur.execute(sql).fetchall()

        if not tables:
            print("В базе данных нет таблиц.")
            return

```

```

print("\nДоступные таблицы:")
for i, table in enumerate(tables, 1):
    print(f"{i}. {table[0]}")

try:
    choice = input(f"\nВыберите номер таблицы (1-{len(tables)}) или введите 0 для возврата: ")
    if choice == '0':
        return

    table_num = int(choice) - 1
    if table_num < 0 or table_num >= len(tables):
        print("Неверный номер.")
        return

    tblname = tables[table_num][0]
except ValueError:
    print("Ошибка: введите число.")
    return

# Получаем структуру таблицы
print(f"\nСтруктура таблицы '{tblname}':")
cur.execute(f"PRAGMA table_info({tblname})")
columns = cur.fetchall()
for col in columns:
    print(f"  {col[1]} ({col[2]})")

# Спрашиваем, сколько строк показывать
while True:
    try:
        limit_input = input("\nВведите число (0 - все): ")
    except:
        limit_input = ""

    # Если пустой ввод - спрашиваем снова
    if not limit_input:
        print("Пожалуйста, введите число!")
        continue

    limit = int(limit_input)

    if limit < 0:

```

```

        print("Число не может быть отрицательным!")
        continue

    if limit == 0:
        print("Показаны все строки таблицы")
    else:
        print(f"Показаны первые {limit} строк")

    break # Выход из цикла при успешном вводе

except ValueError:
    print("Ошибка! Введите целое число (например:
10 или 0)")

    # Формируем SQL запрос
    if limit == 0:
        sql = f"SELECT * FROM {tblname}"
    else:
        sql = f"SELECT * FROM {tblname} LIMIT {limit}"

    with con:
        data = cur.execute(sql).fetchall()

        # Получаем общее количество записей
        sql_count = f"SELECT COUNT(*) FROM {tblname}"
        with con:
            total_count = cur.execute(sql_count).fetchone()[0]

        if limit > 0 and limit < total_count:
            print(f"\nПервые {limit} записей из таблицы:
{tblname} (всего записей: {total_count})")
        else:
            print(f"\nТаблица: {tblname} (всего записей:
{total_count})")

    print("\n" * 100)
    if data:
        for row in data:
            print(row)
    else:

```

```

        print("Таблица пуста.")

except sqlite3.Error as e:
    print(f"Ошибка базы данных: {e}")

def main():
    dbname = ''
    while True:
        dbname = input('Укажите имя файла: ').strip()

        if os.path.isfile(dbname):
            break
        else:
            print(f'Нет такого файла!')

    # Подключение к БД
    con = sqlite3.connect(dbname)
    cur = con.cursor()

    # Создание учебной таблицы если не существует
    create_table(cur)
    con.commit()

    while True:
        print("МЕНЮ")
        print("1. Работа с учебной таблицей  
educational_information")
        print("2. Процент преподавателей с учеными степенями")
        print("3. Распределение ведущих вузов по округам")
        print("4. Показать таблицы из БД VUZ")
        print("5. Выход")

        main_choice = input("Выберите раздел меню: ")

        if main_choice == '1':
            # Подменю для работы с учебной таблицей
            tblname = 'educational_information'
            while True:
                print("\n" + "=" * 50)

```

```

print("МЕНЮ УПРАВЛЕНИЯ БАЗОЙ ДАННЫХ")
print("=" * 50)
print("1. Показать список полей таблицы")
print("2. Показать все записи таблицы")
print("3. Сохранить таблицу в файл")
print("4. Фильтровать записи")
print("5. Удалить записи")
print("6. Обновить записи")
print("7. Добавить запись")
print("8. Вернуться в главное меню")
print("=" * 50)

choice = input("Выберите действие: ")

if choice == '1':
    display_columns(cur, tblname)

elif choice == '2':
    dan = select_cmd(con, cur, tblname)
    nzap = len(dan)
    print(f'\nТаблица: {tblname} из БД
{dbname}')

    print(f"Количество записей: {nzap}")
    print("-" * 80)
    for i in range(nzap):
        print(dan[i])
    if nzap == 0:
        print("(таблица пуста)")

elif choice == '3':
    dan = select_cmd(con, cur, tblname)
    if dan:
        filename = input("Введите имя файла для
сохранения: ")
        save_to_file(dan, tblname, filename)
    else:
        print("Таблица пуста, сохранять
ничего.")

elif choice == '4':

```

```
        filter_data(con, cur, tblname)

    elif choice == '5':
        delete_data(con, cur, tblname)

    elif choice == '6':
        update_data(con, cur, tblname)

    elif choice == '7':
        add_data(con, cur, tblname)

    elif choice == '8':
        break

    else:
        print("Неверный выбор")

elif main_choice == '2':
    task1_vuz(con, cur)

elif main_choice == '3':
    task2_vuz(con, cur)

elif main_choice == '4':
    show_any_table(con, cur)

elif main_choice == '5':
    print("Выход из программы")
    break

else:
    print("Неверный выбор")

# Заккрытие соединения
cur.close()
con.close()

if __name__ == '__main__':
    main()
```

Вывод

В ходе научно-исследовательской работы мы научились работать с реляционными базами данных с использованием языка Python. На этапе 2 выполнены базовые операции: проверка доступности библиотеки sqlite3, создание тестовой базы данных bd1.sqlite с таблицами stud и sessija, отработка операций вставки, чтения, обновления и удаления данных. Разработана программа test3.py для просмотра содержимого таблиц с обработкой ошибок, а также интерактивное приложение с меню для управления базой данных bd2.sqlite, поддерживающее просмотр структуры, фильтрацию, добавление, изменение, удаление записей и сохранение данных в файл. На этапе 3 создана программа с консольным меню для работы с учебными данными, реализующая полный цикл операций CRUD. На этапе 4 разработана программа для анализа данных о вузах России из базы VUZ.sqlite, выполняющая два задания: фильтрацию вузов по проценту преподавателей с учёными степенями и статистику распределения ведущих вузов по федеральным округам. Программа корректно обрабатывает пользовательский ввод, использует параметризованные запросы для безопасности, предоставляет интерактивное меню и выводит результаты в структурированном виде.